

AxelarApp

Contracts

ENGAGEMENT	SECURITY REVIEW
REVISION	1.1
DELIVERED	2026-08-13
LEAD AUDITOR	Naoki Yoshida

Contents

1. Document Revisions.....	3
2. Overview.....	4
2.1. ack3.....	4
2.2. Review Methodology.....	5
2.3. Finding Classification	6
2.4. Review Team	8
2.5. Disclaimer	8
3. Executive Summary.....	9
Revision 1.0.....	9
Revision 1.1.....	11
4. Findings Summary	13
Report Revision 1.0	15
Revision Team	15
System Overview.....	15
Trust Model	15
Findings.....	15
Report Revision 1.1	51
Revision Team	51
System Overview.....	51
Trust Model	51
Appendix A: How to cite.....	52

1. Document Revisions

1.0	Final Report	July 22, 2026
1.1	Fix Review	August 13, 2026

2. Overview

This document presents the findings of our review.

2.1. ack3

[ack3](#) is an in-house team of security researchers performing audits focused on manual code reviews with extensive fuzz testing and AI-assisted workflows for Ethereum and Solana. The ack3 team is trusted by top-tier organizations in web3, including Lido, Safe, and Axelar.

We develop the open-source security and developer tooling [Wake](#), which powers the fuzzing, static analysis, and graph-based review on our audits and helps auditors discover hardly recognizable edge-case vulnerabilities.

Our team teaches blockchain security at the [Czech Technical University in Prague](#), led by founder Josef Gattermayer, Ph.D.

For ack3, security is a long-term partnership: we work alongside our clients from design through post-deployment.

2.2. Review Methodology

We begin each review by mapping the codebase and the interactions between contracts and external components using [Wake](#)'s graph tooling. We work with the provided documentation and materials to discover discrepancies between the expected behavior and the actual implementation. Where applicable, we build a mathematical model of the protocol to cross-compare with the code and identify subtle bugs and inconsistencies.

At the start of the review, we launch an automated [ack3](#) AI scan pinned to the scoped commits, running in parallel with the manual review. The scan provides additional coverage and new leads for the team to analyze; each scan finding is manually reviewed by the team.

During the review, we apply AI-assisted workflows and skills that support codebase onboarding and navigation, combined with Wake's static analyzer and language server for context-aware code analysis.

We launch AI agents to build on our vulnerability hypotheses and expand the attack surface with new escalation paths. All findings are manually reviewed and validated by the team. Where applicable, we confirm findings and verify the exact behavior of the code with unit tests and fuzzing. Where the engagement includes a fuzzing campaign, it is manually guided by an auditor; the fuzz tests themselves are AI-generated.

Once sufficient context is gathered, we search for deeper, chained issues based on the attack vectors discovered during review. This includes combining smaller weaknesses, testing trust-boundary assumptions, and evaluating whether unexpected behavior can lead to practical impact.

The validated results are documented in the report: findings with severity, affected components, impact, and recommended remediation.

2.3. Finding Classification

The Severity of each finding combines two sub-ratings, Impact and Likelihood, and ranges from Info to Critical.

If we find a scenario in which an issue is exploitable, it is assigned an impact rating of High, Medium, or Low, based on its consequences for the system. If we find no way to exploit it, or the issue is only exploitable given a change in configuration (system settings or parameters, such as deployment scripts, compiler configurations, or multi-signature wallets for owners) or in the codebase, it is assigned Warning or Info.

Low to High impact issues also have a Likelihood, which measures the probability of exploitation at runtime.

Severity

		LIKELIHOOD			
		HIGH	MEDIUM	LOW	N/A
IMPACT	HIGH	Critical	High	Medium	-
	MEDIUM	High	Medium	Low	-
	LOW	Medium	Low	Low	-
	WARNING	-	-	-	Warning
	INFO	-	-	-	Info

Table 1. Severity of findings

Impact

- **High** - Code that activates the issue leads to undefined or catastrophic consequences for the system.
- **Medium** - Code that activates the issue results in serious consequences.
- **Low** - Code that activates the issue has consequences that are recoverable or do not jeopardize the system's normal operation.
- **Warning** - The issue cannot be exploited given the current code or configuration, but could become a security vulnerability if either changed. If we have not found a way to exploit the issue given the time constraints, it may be marked as Warning or higher, based on our best estimate of whether it is currently exploitable.
- **Info** - The issue is on the borderline between code quality and security. Examples include insufficient logging for critical operations, or issues that would become security-related if code or configuration changed.

Likelihood

- **High** - The issue is exploitable by virtually anyone under any circumstance.
- **Medium** - Exploiting the issue requires non-trivial preconditions.
- **Low** - Exploiting the issue requires strict preconditions.

2.4. Review Team

For a revision’s authors, see the “Revision team” section in that revision’s chapter.

NAME	POSITION
Naoki Yoshida	Lead Auditor
Jan Kalivoda	Auditor
Michal Převrátíl	Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

2.5. Disclaimer

We have made our best effort to find all vulnerabilities in the system; however, our findings should not be considered a complete list of issues. Statements in this document should not be interpreted as investment or legal advice, nor should the authors be held accountable for decisions based on them.

3. Executive Summary

AxelarApp is a cross-chain application built on Axelar that lets users move tokens across chains while running execution logic on both the source and destination chains. A single call can perform a source-side action such as a swap, bridge the resulting token through the Interchain Token Service or the Axelar gateway, and run a destination-side action on delivery, using the `ExpressExecutable` to settle transfers faster.

Revision 1.0

Axelar engaged ack3 to perform a security review of AxelarApp Contracts with a total time donation of 5 engineering days in a period between July 14 and July 20, 2026, with Naoki Yoshida as the lead reviewer.

The review covered two repositories. In the `axelar-app` repository, we reviewed the contracts under `apps/contracts/contracts` at commit `8d5e2fe`^[1].

- `apps/contracts/contracts/AxelarApp.sol`
- `apps/contracts/contracts/AxelarAppProxy.sol`
- `apps/contracts/contracts/AxelarMulticall.sol`
- `apps/contracts/contracts/AxelarPermit2.sol`
- `apps/contracts/contracts/interfaces/IAxelarApp.sol`
- `apps/contracts/contracts/interfaces/IAxelarGasService.sol`
- `apps/contracts/contracts/interfaces/IAxelarMulticall.sol`
- `apps/contracts/contracts/interfaces/IInterchainTokenService.sol`
- `apps/contracts/contracts/interfaces/ISignatureTransfer.sol`
- `apps/contracts/contracts/libraries/Utils.sol`

In the `axelar-gmp-sdk-solidity` repository, we reviewed the following files at

commit [00682b6^{\[2\]}](#):

- `contracts/express/AxelarExpressExecutableWithToken.sol`
- `contracts/express/ExpressExecutorTracker.sol`
- `contracts/executable/AxelarExecutableWithToken.sol`
- `contracts/executable/AxelarExecutable.sol`
- `contracts/libs/SafeTransfer.sol`

We began our review using static analysis tools, including [Wake](#). We then took a deep dive into the logic of the contracts. For testing and writing exploit scenarios, we used the [Wake](#) testing framework. Because AxelarApp custodies value only transiently while bridging it across chains, we focused on the paths where value enters, moves between the source and destination legs, and leaves the contracts. During the review, we paid special attention to:

- the no-balance invariant, that AxelarApp and AxelarMulticall never retain token or native balances between transactions;
- conservation of value across the source and destination legs of a cross-chain transfer;
- states in which bridged funds can become stranded or unrecoverable on the destination, including flow- and mint-limit rejection and failing retries;
- compatibility with non-standard tokens, such as fee-on-transfer, rebasing, blacklistable, and non-conforming ERC-20 return data;
- the correctness of both the ITS and gateway send and receive paths, and of the express-execution path;
- the multicall executor, including arbitrary-call handling, its reentrancy guard, and calldata parameter splicing;
- completeness of the pause mechanism across the send, receive, and express paths;

- trust boundaries around caller-supplied routing, recipients, and ITS operators; and
- ensuring access controls are not too relaxed or too strict.

Our review resulted in 16 findings, ranging from Info to High severity. The most severe is High severity and can leave bridged funds stuck cross-chain. In [H1: Unsafe leftover-recipient refunds](#), refunds to the leftoverRecipient use a reverting safeTransfer, so a non-receivable (e.g. blacklisted) recipient reverts the entire receive hook and rolls back the destination credit with no on-chain recovery path.

ack3 recommends Axelar:

- mitigate the identified griefing attack vectors;
- read and review the complete report; and
- address all identified issues.

See [Report Revision 1.0](#) for the system overview and trust model.

Revision 1.1

Axelar engaged ack3 to perform a fix review of the findings from [Revision 1.0](#).

The review was performed between August 3 and August 3, 2026 on the commit 786a50c^[3]. The scope was the apps/contracts directory of the axelar-app repository. The axelar-gmp-sdk-solidity dependency was not re-reviewed, and its pins are unchanged.

From the reported 16 findings, 10 issues were fixed and 6 issues were acknowledged.

No new findings were reported.

See [Report Revision 1.1](#) for more details.

[1] full commit hash: 8d5e2fe59829fbda7b2d4f02403816f9e708d772

[2] full commit hash: 00682b6c3db0cc922ec0c4ea3791852c93d7ae31, link to [commit](#)

[3] full commit hash: 786a50c0e1db30a633b6e7cea15f069db1717005

4. Findings Summary

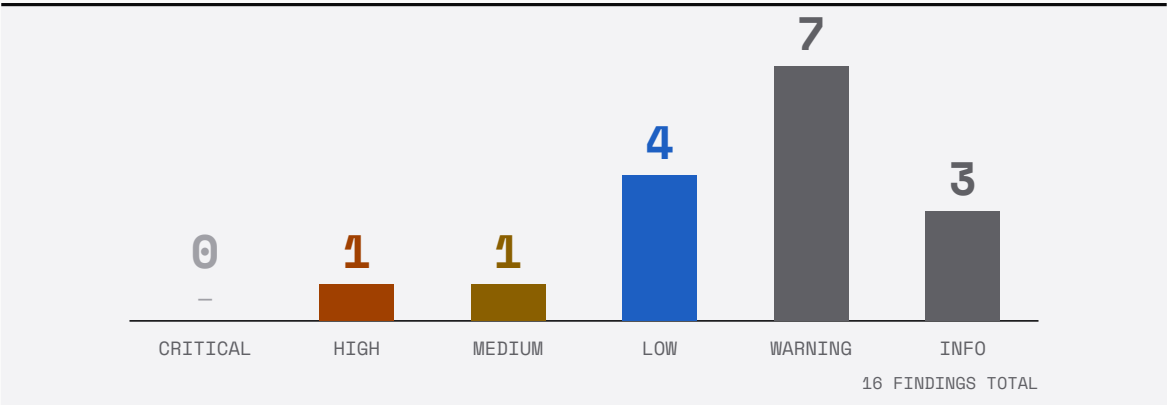


Figure 1. Findings by Severity

FINDING TITLE	SEVERITY	REVISION	STATUS
H1: Unsafe leftover-recipient refunds	HIGH	1.0	Acknowledged
M1: Irreversible refund through shared multical guard	MEDIUM	1.0	Fixed
L1: Incorrect FullTokenBalance splice documentation	LOW	1.0	Fixed
L2: Shared executor state poisoning	LOW	1.0	Acknowledged
L3: Uncapped cross-chain transfer amount	LOW	1.0	Acknowledged
L4: Implicit Cancun hardfork requirement	LOW	1.0	Fixed

FINDING TITLE	SEVERITY	REVISION	STATUS
W1: Unreachable ITS unsupported-token error	WARNING	1.0	Fixed
W2: Malformed ERC20 return data in tryTransfer	WARNING	1.0	Acknowledged
W3: Unbounded gateway gas payment	WARNING	1.0	Acknowledged
W4: ITS refunds to transaction origin	WARNING	1.0	Fixed
W5: Unoptimized deployment build	WARNING	1.0	Fixed
W6: Stranded native value in express token execution	WARNING	1.0	Fixed
W7: Unpausable empty-data ITS delivery	WARNING	1.0	Acknowledged
I1: Undocumented native token sentinel	INFO	1.0	Fixed
I2: Inverted service availability documentation	INFO	1.0	Fixed
I3: Deprecated ITS transfer overload	INFO	1.0	Fixed

Table 2. Table of Findings

Report Revision 1.0

Revision Team

NAME	POSITION
Naoki Yoshida	Lead Auditor
Jan Kalivoda	Auditor
Michal Převrátíl	Auditor
Josef Gattermayer, Ph.D.	Audit Supervisor

System Overview

AxelarApp Contracts lets users prepare tokens for a cross-chain transfer and trigger execution on the destination chain. It covers both the execution on the source chain and the transfer-and-execute flow on the destination chain, and it leverages the `ExpressExecutable` to complete these operations faster.

Trust Model

Users must trust each interchain token manager, and the Interchain Token Service (ITS) in particular. Users must supply `calldata` that does not leave tokens stranded in the `AxelarApp` and `AxelarMulticall` contracts. Users must specify a valid `leftoverRecipient`: the receive path sends the delivered tokens there whenever the recipient cannot receive them or the destination multicall fails, and the contract does not validate it.

Findings

This section lists the findings from this revision; for all findings, see the [Findings Summary](#).

H1: Unsafe leftover-recipient refunds

HIGH SEVERITY ISSUE

Impact:	High	Likelihood:	Medium
Type:	Denial of service	Discoverer:	Human
Target:	AxelarApp.sol		

Description

`_deliver` uses `Utils.tryTransfer` for the primary recipient, but refunds and dust sweeps to `leftoverRecipient` use reverting `safeTransfer` calls. For blacklistable or otherwise non-receivable tokens, these calls revert instead of falling back.

Listing 1. Excerpt from `AxelarApp._deliver`

```
289 if (!Utils.tryTransfer(token, recipient, amount)) {
290     // We assume that leftoverRecipient is always a valid address
291     IERC20(token).safeTransfer(leftoverRecipient, amount);
```

Listing 2. Excerpt from `AxelarApp._deliver`

```
306 if (remaining > 0) IERC20(token).safeTransfer(leftoverRecipient,
    remaining);
```

Listing 3. Excerpt from `AxelarApp._deliver`

```
311 IERC20(token).safeTransfer(leftoverRecipient, amount);
```

The receive hooks call `_deliver` after ITS or gateway delivery in the same transaction. If any `leftoverRecipient` transfer reverts, the whole receive reverts, rolling back the destination credit and command execution state. Because the cross-chain payload fixes the same `leftoverRecipient`, relayer

retries fail deterministically, and rescueFunds cannot recover the amount because no balance persists in AxelarApp after rollback.

Exploit scenario

Alice sends a supported blacklistable token with a valid send-and-call payload that specifies Bob as leftoverRecipient. Before destination execution, Bob is blacklisted by the token issuer.

The destination multicall reverts, so `_deliver` enters the refund path and calls `safeTransfer` to Bob. The token rejects the transfer, the receive hook reverts, and each retry uses the same payload and reverts the same way. Alice's bridged amount remains stuck with no on-chain recovery path.

Recommendation

We recommend to acknowledge and remain aware of this behavior. The source and destination chains run separate contract deployments, so a bridged asset such as USDT can have different fee-on-transfer or blacklist behavior on each chain. So care is needed over which tokens are enabled for send-and-call flows, as a non-receivable recipient and leftoverRecipient can leave an inbound delivery permanently stuck.

Acknowledgment 1.1

The finding is acknowledged with the leftoverRecipient requirement documented in `IAxelarApp`, `_deliver`, and `DEPLOYMENT.md`.

[Go back to Findings Summary](#)

M1: Irreversible refund through shared multicall guard

MEDIUM SEVERITY ISSUE

Impact:	Medium	Likelihood:	Medium
Type:	Denial of service	Discoverer:	AI
Target:	AxelarMulticall.sol, AxelarApp.sol, AxelarExpressExecutableWithToken.sol		

Description

AxelarMulticall.run is externally callable and uses one contract-wide running flag. An attacker can enter run with a call that invokes an approved destination delivery, causing the delivery's non-empty destinationPayload to re-enter the same run and deterministically revert with AlreadyRunning.

Listing 4. Excerpt from AxelarMulticall

```
25 modifier nonReentrant() {
26     if (running) revert AlreadyRunning();
27     running = true;
28     _;
29     running = false;
30 }
31
32 /// @inheritdoc IAxelarMulticall
33 function run(bytes calldata encodedCalls) external payable nonReentrant {
```

executeWithToken validates the gateway message before the application hook runs. If _deliver then catches the nested multicall revert, the hook returns normally and the approved command is consumed.

Listing 5. Excerpt from [AxelarExpressExecutableWithToken.executeWithToken](#)

```
55 !gatewayWithToken().validateContractCallAndMint(
56     commandId,
57     sourceChain,
```

```
58     sourceAddress,  
59     payloadHash,  
60     tokenSymbol,  
61     amount  
62 )  
63 ) revert NotApprovedByGateway();
```

Listing 6. Excerpt from AxelarApp

```
300 Utils.smartApprove(token, address(multicall), amount);  
301 try multicall.run(destinationPayload) {  
302     // Enforce the no-balance invariant: the payload must consume the  
303     // delivered token. A successful  
304     // but non-consuming payload would otherwise leave `token` here behind  
305     // the standing multicall  
306     // allowance, so sweep any remainder to leftoverRecipient.  
307     uint256 remaining = IERC20(token).balanceOf(address(this));  
308     if (remaining > 0) IERC20(token).safeTransfer(leftoverRecipient,  
309     remaining);  
310     emit SendAndCallSucceeded(keccak256(envelope));  
311 } catch (bytes memory reason) {  
312     // We assume that leftoverRecipient is always a valid address since it  
313     // is added as a fallback  
314     // and should be checked by the payload provider before being  
315     // specified.  
316     IERC20(token).safeTransfer(leftoverRecipient, amount);  
317     emit SendAndCallFailed(keccak256(envelope), leftoverRecipient,  
318     reason);
```

As a result, the bridged amount is refunded to the authenticated leftoverRecipient, but the intended destination operations are permanently skipped instead of remaining retryable. This is a deterministic denial of service for approved messages with non-empty destination payloads when the attacker wins the permissionless execution race.

Exploit scenario

Alice sends a cross-chain message whose destination payload should swap or forward the received tokens. After the gateway approves the message but before the honest relayer executes it, Bob calls `AxelarMulticall.run` with a `Default` call to `executeWithToken` for Alice's approved command. The outer run

sets running, `executeWithToken` validates the command, and `_deliver` tries to execute Alice's destination payload through the same `multicall.run`. The nested call reverts with `AlreadyRunning`, `_deliver` catches the revert, refunds Alice's tokens to `leftoverRecipient`, and returns successfully. The gateway command is consumed, so Alice's intended destination payload can no longer be executed.

Recommendation

Do not share the same `AxelarMulticall` reentrancy guard between untrusted top-level batches and destination deliveries. Use a separate executor or guard context for `_deliver`, or make `AlreadyRunning` bubble as a top-level revert so the gateway message remains retryable instead of entering the refund path.

Fix 1.1

The finding is fixed by exposing the guard check as a separate `AxelarMulticall.ensureNotRunning` view, which `_deliver` calls outside the `try` block, so a nested delivery reverts the whole receive and the gateway message remains retryable.

[Go back to Findings Summary](#)

L1: Incorrect FullTokenBalance splice documentation

LOW SEVERITY ISSUE

Impact:	Low	Likelihood:	Low
Type:	Code quality	Discoverer:	Human
Target:	IAxelarMulticall.sol		

Description

IAxelarMulticall describes the FullTokenBalance splice position as a calldata offset.

Listing 7. Excerpt from IAxelarMulticall

```
19 /// @notice Splice this contract's full balance of the token encoded in  
   'payload' into 'callData'  
20 ///   at the encoded offset before calling, chains a prior step's output  
   into this step's input.
```

Listing 8. Excerpt from IAxelarMulticall

```
33 /// @param payload Auxiliary data interpreted per 'callType' (e.g. the  
   token address + calldata  
34 ///   offset for a 'FullTokenBalance' splice). Unused for 'Default'.
```

AxelarMulticall instead treats the value as a zero-based 32-byte argument-slot index and writes at `callData + 36 + parameterPosition * 32`.

Listing 9. Excerpt from AxelarMulticall._setCallDataParameter

```
66 if (callData.length < 36 || parameterPosition > (callData.length - 36) /  
    32) {  
67     revert ParameterOutOfBounds(parameterPosition, callData.length);  
68 }  
69 assembly {  
70     mstore(add(callData, add(36, mul(parameterPosition, 32))), value)
```

Payload builders that follow the interface `NatSpec` and pass byte offsets instead of slot indexes produce malformed payloads that revert with `ParameterOutOfBounds` or `CallFailed`. This is memory-safe because the bounds check prevents out-of-bounds writes, but the contradictory documentation can prevent integrations from constructing valid `FullTokenBalance` splices.

Exploit scenario

An integrator builds a `FullTokenBalance` call by following the `IAxelarMulticall` `NatSpec` and encodes the splice position as a `calldata` byte offset (for example, 36 for the first argument after the selector).

`AxelarMulticall._setCallDataParameter` instead treats that value as a 32-byte argument-slot index and computes $\text{calldata} + 36 + 36 * 32$, which lands outside the buffer. The call reverts with `ParameterOutOfBounds` (or `CallFailed`), so the integrator's source or destination multicall fails until they discover the undocumented slot-index convention by reading the implementation.

Recommendation

Update the `FullTokenBalance` `NatSpec` to describe payload as the token address plus a zero-based 32-byte argument-slot index, where 0 is the first argument after the selector.

Fix 1.1

The finding is fixed by correcting the `IAxelarMulticall` documentation to describe the `FullTokenBalance` payload as the token address plus a zero-based 32-byte argument-slot index.

[Go back to Findings Summary](#)

L2: Shared executor state poisoning

LOW SEVERITY ISSUE

Impact:	Medium	Likelihood:	Low
Type:	Trust model	Discoverer:	Human
Target:	AxelarMulticall.sol, AxelarApp.sol		

Description

AxelarMulticall.run is permissionless and forwards arbitrary calls from the shared AxelarMulticall address.

Listing 10. Excerpt from AxelarMulticall

```
33 function run(bytes calldata encodedCalls) external payable nonReentrant {
```

Listing 11. Excerpt from AxelarMulticall.run

```
54 (bool success, bytes memory data) = call.target.call{value:
    call.value}(call.callData);
55 if (!success) revert CallFailed(i, data);
```

The transient running flag only blocks nested calls to run. It does not stop an attacker from using the shared executor identity to create persistent state in external contracts, such as an ERC-1820 ERC777TokensRecipient registration for the multicall and a token allowance from the multicall to the attacker.

For source routes, AxelarApp.callAndSend pulls the input token before running the source payload, and routes with a source payload send that input directly to multicall.

Listing 12. Excerpt from AxelarApp

```
99 _pullToken(params, outputToken);
```

```
100
101 if (params.sourcePayload.length > 0) {
102     _runSourcePayload(params);
```

Listing 13. Excerpt from AxelarApp._pullToken

```
153 address pullRecipient = fundsMulticallDirectly ? address(multicall) :
    address(this);
154 if (params.intakeMode == IntakeMode.Permit2) {
155     _pullViaPermit2To(params.permit, params.amount, pullRecipient,
        params.signature);
156 } else {
157     IERC20(inputToken).safeTransferFrom(msg.sender, pullRecipient,
        params.amount);
```

For destination routes, CollectTokenBalance transfers the token into the multicall during run.

Listing 14. Excerpt from AxelarMulticall

```
46 } else if (call.callType == CallType.CollectTokenBalance) {
47     // Intake: pull the caller's full balance of 'token' in, then skip the
        target call.
48     address token = abi.decode(call.payload, (address));
49     IERC20(token).safeTransferFrom(msg.sender, address(this),
        IERC20(token).balanceOf(msg.sender));
```

A transfer-hook token can therefore invoke the planted recipient hook when the multicall receives the victim's tokens. The hook can transfer tokens out through the pre-set allowance without calling run, so AlreadyRunning is not reached on source pre-funding and is irrelevant during destination collection. If the route still succeeds, the hook transfer is committed and the victim receives less output. The same durable-state primitive can also poison non-zero allowances and make later USDT-style approve calls from the multicall revert.

Exploit scenario

Eve calls AxelarMulticall.run with a batch that registers Eve's

ERC777TokensRecipient hook for AxelarMulticall in ERC-1820 and approves Eve to spend the hook token from AxelarMulticall.

Alice later routes that hook token through AxelarApp with a source payload that consumes the multicall's live balance. When `_pullToken` pre-funds AxelarMulticall, Eve's hook runs before `run` and transfers part of Alice's tokens to Eve through the standing allowance. The source payload uses the remaining balance and succeeds within Alice's slippage tolerance, so the theft is not reverted.

Recommendation

Execute untrusted batches from an isolated address, such as a minimal proxy clone created per `run`, so external caller-scoped state cannot persist across users. If a shared executor is retained, remove public arbitrary execution or validate allowed targets and selectors, set the executor's ERC-1820 manager to a trusted no-op manager, and zero allowances before and after approvals with a `forceApprove`-style sequence.

Acknowledgment 1.1

The finding is acknowledged.

[Go back to Findings Summary](#)

L3: Uncapped cross-chain transfer amount

LOW SEVERITY ISSUE

Impact:	Medium	Likelihood:	Low
Type:	Denial of service	Discoverer:	Human
Target:	AxelarApp.sol		

Description

`AxelarApp.callAndSend` sends the contract's full `outputToken` balance after the optional source payload. This supports swaps with unknown exact output, but the caller cannot cap the amount and any unsolicited balance or donation is included.

Listing 15. Excerpt from `AxelarApp.callAndSend`

```
107 uint256 sendAmount = IERC20(outputToken).balanceOf(address(this));
```

The uncapped value is used for both `InterchainToken` and `GatewayToken` sends.

Listing 16. Excerpt from `AxelarApp.callAndSend`

```
110 if (params.kind == SendKind.InterchainToken) {
111     _sendInterchainToken(params, outputToken, sendAmount);
112 } else if (params.kind == SendKind.GatewayToken) {
113     _sendGatewayToken(params, outputToken, sendAmount);
```

For ITS tokens, the inflated amount can exceed the source or destination flow limit. For gateway tokens, the same issue applies to destination mint limits where they are enabled. If the source leg accepts the transfer and the destination limit rejects it, the message keeps the committed amount and retries fail while that limit stays below the amount. The same delay can occur without a donation when the source accepts an amount that is above a lower

destination limit.

This requires a limit to be configured on the affected token and the inflated amount to actually cross it, so it is unlikely to occur in normal operation. The transfer is delayed rather than lost: where the failure is an epoch-based cumulative flow check, a later epoch makes the message executable again, and otherwise the relevant limit can be raised. Sustaining the denial requires an attacker to donate enough to fill the remaining destination capacity in every epoch, so the cost grows with the duration of the attack.

Exploit scenario

Alice uses `callAndSend` with a source swap and bridges the resulting `outputToken`. Bob donates the same `outputToken` to `AxelarApp` before Alice's transaction, so `sendAmount` includes Alice's swap output and Bob's donation. The source bridge action is committed, but the destination flow or mint limit rejects the inflated amount before delivery to Alice's destination payload. Each retry uses the same committed amount, so Alice's funds stay undelivered until the destination epoch capacity resets or the limit is raised.

Recommendation

Do not rely on the full contract balance as an unbounded cross-chain transfer amount. Because a source swap can make the exact output unknown before execution, add an optional `maxSendAmount` and revert when the produced balance exceeds it. Integrators should also check current source and destination flow or mint limits before sending and choose a bound that cannot exceed the destination limit or available epoch capacity.

Acknowledgment 1.1

The finding is acknowledged.

[Go back to Findings Summary](#)

L4: Implicit Cancun hardfork requirement

LOW SEVERITY ISSUE

Impact:	Medium	Likelihood:	Low
Type:	Configuration	Discoverer:	Human
Target:	AxelarMulticall.sol, foundry.toml, mainnet.json		

Description

AxelarMulticall stores its reentrancy guard in transient storage. Solidity lowers this to TLOAD and TSTORE, so every `run()` execution requires support for [EIP-1153](#).

Listing 17. Excerpt from AxelarMulticall

```
20 /// @dev Transient reentrancy flag; auto-cleared at the end of the  
    transaction.  
21 bool private transient running;  
22  
23 /// @dev Reverts re-entrant entry into `run` so a mid-call control handoff  
    (e.g. a token or pool hook)  
24 /// cannot drain the working balance; sequential top-level calls in one  
    transaction are still allowed.  
25 modifier nonReentrant() {  
26     if (running) revert AlreadyRunning();  
27     running = true;  
28     _;  
29     running = false;
```

The project pins `solc_version` but not `evm_version`, while the shipped chain matrix still includes Kava (`"chainType": "evm", chainId 2222`) as an EVM target.

With Shanghai-or-newer bytecode, pre-Shanghai targets can reject deployment when creation code executes `PUSH0`. On Shanghai-but-not-Cancun targets, deployment can succeed while every `run()` reverts as soon as the guard executes. This can silently brick source swaps and inbound

deliveries on an incompatible supported chain.

Exploit scenario

The team deploys the contracts to a supported chain whose EVM predates Cancun, or that otherwise lacks [EIP-1153](#) support, while the deployment scripts pin only the compiler version. The artifacts are built with Cancun-era bytecode, so deployment on a Shanghai-but-not-Cancun target succeeds without warning. A user then calls `callAndSend` with a source payload, or an inbound message triggers a destination delivery; either path invokes `AxelarMulticall.run`, whose reentrancy guard executes `TSTORE` and `TLOAD`. The chain rejects the transient-storage opcodes, so every `run()` reverts, bricking source swaps and inbound deliveries on that chain until the contracts are redeployed with a compatible guard.

Recommendation

Choose the supported hardfork boundary explicitly. If Cancun is required, pin `evm_version = "cancun"`, document the minimum hardfork, remove non-Cancun chains such as Kava from the matrix, and add a deployment preflight that aborts when `PUSH0` or `TLOAD` probes fail. If pre-Cancun chains must remain supported, replace the transient guard with a storage-based guard and pin `evm_version` to the lowest supported fork.

Fix 1.1

The finding is fixed by replacing the transient reentrancy guard with a storage-based guard and pinning the hardfork to `paris` in both `foundry.toml` and `scripts/compile.ts`, so `run` no longer requires [EIP-1153](#) or `PUSH0`.

[Go back to Findings Summary](#)

W1: Unreachable ITS unsupported-token error

Impact:	Warning	Likelihood:	N/A
Type:	Code quality	Discoverer:	AI
Target:	AxelarApp.sol, Mocks.sol		

Description

AxelarApp._resolveOutputToken calls ITS registeredTokenAddress and then expects an unregistered tokenId to be represented as address(0).

Listing 18. Excerpt from AxelarApp._resolveOutputToken

```
125 address interchainToken =  
126     IInterchainTokenService(interchainTokenService).registeredTokenAddress(par  
    ams.interchainSend.tokenId);  
127 if (interchainToken == address(0)) revert  
    UnsupportedInterchainToken(params.interchainSend.tokenId);
```

The pinned @axelar-network/interchain-token-service version is 2.2.0, where registeredTokenAddress resolves through deployedTokenManager and reverts with TokenManagerDoesNotExist when no token manager is deployed. Because _resolveOutputToken does not catch that revert, the zero-address check and the intended UnsupportedInterchainToken error are unreachable against the real ITS implementation. Integrators therefore see the ITS error instead of the application’s documented error surface.

The test mock hides this divergence by returning the zero address for unknown token IDs.

Listing 19. Excerpt from MockIts.registeredTokenAddress

```
182 function registeredTokenAddress(bytes32 tokenId) external view returns  
    (address) {  
183     return _registered[tokenId];
```

```
184 }
```

This has no direct funds-at-risk impact because the revert occurs before tokens are pulled, but it leaves dead production code and inaccurate tests.

Recommendation

Either remove the zero-address check and rely on the ITS revert, or wrap `registeredTokenAddress` in `try/catch` and translate the ITS failure to `UnsupportedInterchainToken`. Update `MockIts` to revert for unregistered token IDs so tests match ITS 2.2.0.

Fix 1.1

The finding is fixed by removing the unreachable zero-address check and the `UnsupportedInterchainToken` error, so the ITS `TokenManagerDoesNotExist` revert reaches the caller. `MockITS` is updated to revert for unregistered token IDs.

[Go back to Findings Summary](#)

W2: Malformed ERC20 return data in tryTransfer

Impact:	Warning	Likelihood:	N/A
Type:	Data validation	Discoverer:	Human
Target:	Utils.sol, AxelarApp.sol		

Description

Utils.tryTransfer is documented as a non-reverting transfer helper, but it directly decodes every non-empty token return value as bool. Solidity's bool decoder reverts for malformed payloads, such as payloads shorter than an ABI word or non-canonical bool words. As a result, a non-standard ERC20 can make tryTransfer revert instead of returning false.

Listing 20. Excerpt from Utils

```

59 function tryTransfer(address token, address to, uint256 amount) internal
    returns (bool) {
60     if (to == address(0)) return false;
61     (bool ok, bytes memory data) =
        token.call(abi.encodeWithSelector(IERC20.transfer.selector, to, amount));
62     return ok && token.code.length != 0 && (data.length == 0 ||
        abi.decode(data, (bool)));

```

The empty-payload branch of _deliver relies on that false return to route funds to leftoverRecipient. If tryTransfer reverts, the fallback is not reached and the GMP receive reverts instead. This is not reachable with the current supported token set, but it becomes a delivery-bricking risk if malformed-return tokens are supported later.

Listing 21. Excerpt from AxelarApp

```

287 if (destinationPayload.length == 0) {
288     // Deliver to recipient; if it cannot receive, fall back to
        leftoverRecipient
289     if (!Utils.tryTransfer(token, recipient, amount)) {

```



```
290      // We assume that leftoverRecipient is always a valid address
291      IERC20(token).safeTransfer(leftoverRecipient, amount);
```

Recommendation

Decode optional ERC20 return data defensively in `tryTransfer`. Check that the returned data contains a full word and compare the decoded word to the canonical success value without using the `bool` decoder, returning `false` for malformed data.

If malformed-return tokens are intended to be supported end-to-end, also replace the `leftoverRecipient` transfer legs with a revert-safe transfer primitive or explicitly keep such tokens out of scope.

Acknowledgment 1.1

The finding is acknowledged.

[Go back to Findings Summary](#)

W3: Unbounded gateway gas payment

Impact:	Warning	Likelihood:	N/A
Type:	Logic error	Discoverer:	Human
Target:	AxelarApp.sol, IAxelarApp.sol		

Description

The gateway send path pays Axelar gas from the entire native balance left in AxelarApp. Because `_payGas` runs before `refundsResidualNative`, any accidental over-deposit is swept into the gas service and the on-chain residual refund is normally unreachable for GatewayToken sends.

Listing 22. Excerpt from AxelarApp

```
219 function _payGas(CallAndSendParams calldata params, uint256 amount, bytes
    memory payload) private {
220     uint256 gasValue = address(this).balance;
221     if (gasValue == 0) return;
```

Listing 23. Excerpt from AxelarApp

```
63 modifier refundsResidualNative() {
64     _;
65     uint256 residual = address(this).balance;
66     if (residual > 0) {
67         (bool ok,) = payable(msg.sender).call{value: residual}("");
68         if (!ok) revert NativeRefundFailed();
```

This differs from the ITS path, where the caller supplies an explicit `gasValue` and only that amount is forwarded. `GatewaySendParams` has no corresponding gas budget field, so the gas payment is whatever native value remains on the contract after the source-side work.

Listing 24. Excerpt from IAxelarApp

```
89 /// @param gasValue          Native amount paid to ITS as cross-chain gas;  
    any surplus native is refunded.  
90 struct InterchainSendParams {  
91     bytes32 tokenId;  
92     bytes destinationAddress;  
93     uint256 gasValue;  
94 }
```

Listing 25. Excerpt from IAxelarApp

```
101 /// @param gasRefundRecipient    Address refunded any unspent cross-  
    chain gas  
102 /// @param enableExpress        true → express gas (boost); false →  
    standard delivery.  
103 struct GatewaySendParams {  
104     string tokenSymbol;  
105     string destinationContractAddress;  
106     address gasRefundRecipient;  
107     bool enableExpress;  
108 }
```

The gateway path also forwards `gatewaySend.gasRefundRecipient` without validation or a default. If the caller passes `address(0)`, any excess-gas refund depends on an unusable refund recipient rather than the app’s on-chain residual refund.

Listing 26. Excerpt from AxelarApp._payGas

```
225 gasService.payNativeGasForExpressCallWithToken{value: gasValue}{  
226     address(this),  
227     params.destinationChain,  
228     gatewaySend.destinationContractAddress,  
229     payload,  
230     gatewaySend.tokenSymbol,  
231     amount,  
232     gatewaySend.gasRefundRecipient
```

Recommendation

Update gateway sends to:

- add a `gasValue` field to `GatewaySendParams` and cap `_payGas` to that caller-specified budget;
- leave surplus native available for `refundsResidualNative` after the send;
- validate `gatewaySend.gasRefundRecipient != address(0)` or default it to `msg.sender`; and
- clarify the `refundsResidualNative` documentation for gateway sends if gas remains intentionally funded through the residual balance.

Acknowledgment 1.1

The finding is acknowledged.

[Go back to Findings Summary](#)

W4: ITS refunds to transaction origin

Impact:	Warning	Likelihood:	N/A
Type:	Trust model	Discoverer:	AI
Target:	AxelarApp.sol, IAxelarApp.sol		

Description

The ITS send parameters expose only gasValue and do not let the caller choose a gas refund recipient.

Listing 27. Excerpt from IAxelarApp.InterchainSendParams

```
89 /// @param gasValue      Native amount paid to ITS as cross-chain gas;  
any surplus native is refunded.  
90 struct InterchainSendParams {  
91     bytes32 tokenId;  
92     bytes destinationAddress;  
93     uint256 gasValue;  
94 }
```

AxelarApp._sendInterchainToken forwards this value to the six-argument interchainTransfer overload. In ITS v2.2.0, that overload pays Axelar gas with tx.origin as the refund address.

Listing 28. Excerpt from AxelarApp._sendInterchainToken

```
183 IInterchainTokenService(interchainTokenService).interchainTransfer{value:  
    interchainSend.gasValue}{  
184     interchainSend.tokenId,  
185     params.destinationChain,  
186     interchainSend.destinationAddress,  
187     amount,  
188     metadata,  
189     interchainSend.gasValue  
190 };
```

As a result, AxelarApp cannot direct asynchronous unused gas refunds to

`msg.sender` or to the account that funded `gasValue`. This is harmless for direct EOA calls, but smart-wallet, Safe, ERC-4337, or relayed submissions can have a user-controlled contract as `msg.sender` while `tx.origin` is the bundler, executor, or relayer EOA. Any surplus ITS gas refund is then paid to that third party. The `refundsResidualNative` modifier only returns native left in `Axe1arApp` after the call and does not affect later ITS gas-service refunds.

Recommendation

Acknowledge and document this behavior, carefully manage `gasValue` to minimize any surplus refunded to `tx.origin`, and avoid smart-wallet or relayed submissions where refund ownership matters.

Fix 1.1

The finding is fixed by documenting the `tx.origin` gas-refund behavior in `Axe1arApp._sendInterchainToken`.

[Go back to Findings Summary](#)

W5: Unoptimized deployment build

Impact:	Warning	Likelihood:	N/A
Type:	Configuration	Discoverer:	Human
Target:	compile.ts		

Description

The deployment path compiles contracts through `compile.ts`, whose Standard JSON settings only request bytecode output and do not configure optimizer or `evmVersion`. Solidity therefore uses its defaults for the deployed artifacts, including optimizer disabled.

Listing 29. Excerpt from `compile`

```
83 const input = JSON.stringify({
84   language: "Solidity",
85   sources,
86   settings: {
87     outputSelection: { "*": { "*": ["evm.bytecode.object"] } },
88   },
```

The Solidity test path uses Foundry with optimizer enabled.

Listing 30. Excerpt from `foundry`

```
6 optimizer = true
```

As a result, gas-sensitive behavior is validated against optimized bytecode while deployment ships unoptimized bytecode. This reduces test fidelity and can hide deployment-only behavior in code paths that depend on gas costs or compiler code generation.

Recommendation

Use one shared compiler configuration for testing and deployment. Set the same optimizer and optimizerRuns values in `compile.ts`, pin the intended `evmVersion`, and consider deploying the Foundry-produced artifacts directly.

Fix 1.1

The finding is fixed by aligning the deployment compiler settings in `scripts/compile.ts` with the Foundry profile, enabling the optimizer at 200 runs and pinning `evmVersion` to `paris`.

[Go back to Findings Summary](#)

W6: Stranded native value in express token execution

Impact:	Warning	Likelihood:	N/A
Type:	Logic error	Discoverer:	Human
Target:	AxelarExpressExecutableWithToken.sol		

Description

AxelarApp inherits `expressExecuteWithToken`, which is an external payable entry point.

Listing 31. Excerpt from [AxelarExpressExecutableWithToken](#)

```

111 function expressExecuteWithToken(
112     bytes32 commandId,
113     string calldata sourceChain,
114     string calldata sourceAddress,
115     bytes calldata payload,
116     string calldata symbol,
117     uint256 amount
118 ) external payable virtual {

```

The AxelarApp receive override only applies whenNotPaused and forwards to `_deliver`. It does not reject or refund attached native value.

Listing 32. Excerpt from AxelarApp

```

266 function _executeWithToken(
267     bytes32, /* commandId */
268     string calldata, /* sourceChain */
269     string calldata, /* sourceAddress */
270     bytes calldata payload,
271     string calldata tokenSymbol,
272     uint256 amount
273 ) internal override whenNotPaused {
274     _deliver(payload, gatewayWithToken().tokenAddresses(tokenSymbol),
        amount);

```

`_deliver` uses the token amount only and calls the destination multicall

without forwarding native value.

Listing 33. Excerpt from AxelarApp

```
300 Utils.smartApprove(token, address(multicall), amount);  
301 try multicall.run(destinationPayload) {
```

As a result, native value sent with `expressExecuteWithToken{value: ...}` is neither consumed nor returned, and remains in AxelarApp until the owner recovers it with `rescueFunds`.

Recommendation

Reject unexpected native value on the express token receive path.

Fix 1.1

The finding is fixed by reverting with the new `UnexpectedNativeValue` error in `_executeWithToken` when `msg.value` is non-zero, so the express token path can no longer strand native value.

[Go back to Findings Summary](#)

W7: Unpausable empty-data ITS delivery

Impact:	Warning	Likelihood:	N/A
Type:	Logic error	Discoverer:	Human
Target:	AxelarApp.sol		

Description

AxelarApp guards ITS receive handling only in `_executeWithInterchainToken`. However, for an ITS transfer with empty data, ITS credits the destination address and skips the executable callback, so this `whenNotPaused` check is not reached.

Listing 34. Excerpt from AxelarApp

```
251 function _executeWithInterchainToken(  
252     bytes32, /* commandId */  
253     string calldata, /* sourceChain */  
254     bytes calldata, /* sourceAddress */  
255     bytes calldata data,  
256     bytes32, /* tokenId */  
257     address token,  
258     uint256 amount  
259 ) internal override whenNotPaused {  
260     _deliver(data, token, amount);
```

As a result, pausing AxelarApp does not prevent ITS tokens from being delivered directly to the app address. Those tokens become an ambient balance that `callAndSend` later treats as part of the next send amount because it sends the contract's raw `balanceOf`.

Listing 35. Excerpt from AxelarApp.callAndSend

```
105 // Invariant: the contract holds no token balance between transactions, so  
    its full balance of  
106 // `outputToken` is exactly what this call produced.  
107 uint256 sendAmount = IERC20(outputToken).balanceOf(address(this));
```

```
108 if (sendAmount == 0) revert NothingProduced();
```

Recommendation

Acknowledge and document this behavior: empty-data ITS transfers are delivered directly to the destination address and bypass the executable callback, so pausing AxelarApp does not block these inbound deliveries.

Acknowledgment 1.1

The finding is acknowledged with the behavior documented in `DEPLOYMENT.md`.

[Go back to Findings Summary](#)

I1: Undocumented native token sentinel

Impact:	Info	Likelihood:	N/A
Type:	Code quality	Discoverer:	Human
Target:	IAxelarApp.sol, Utils.sol		

Description

IAxelarApp.rescueFunds documents that token may be an ERC20 or native asset, but it does not state how callers select the native path.

Listing 36. Excerpt from IAxelarApp

```
156 /// @notice Recover tokens (ERC20 or native) mistakenly sent to this
    contract.
157 function rescueFunds(address token, address payable to, uint256 amount)
    external;
```

rescueFunds reaches Utils.smartTransfer, which treats native funds only as the internal NATIVE_TOKEN sentinel. address(0) is handled as an ERC20 token address and reverts through the safe-transfer code path, so this fails closed rather than causing silent loss.

Listing 37. Excerpt from Utils.NATIVE_TOKEN

```
25 /// @notice Sentinel address representing the chain's native token in
    'smartTransfer'.
26 address internal constant NATIVE_TOKEN =
    0xEeeeeEeeeEeEeeEeEeEEEEEEEEEEEEEEEEEE;
```

Listing 38. Excerpt from Utils.smartTransfer

```
70 if (token == NATIVE_TOKEN) {
71     (bool ok,) = to.call{value: amount}("");
72     if (!ok) revert NativeTransferFailed();
73 } else {
```

```
74 IERC20(token).safeTransfer(to, amount);
```

Because the sentinel is not exposed through the interface, this creates an admin-runbook footgun and an inconsistency with the vendored SDK convention that uses `address(0)` for native value.

Recommendation

Document the sentinel in `IAxelarApp.rescueFunds`, including that `address(0)` is not native and reverts. Expose a shared native-token constant or explicitly document the deliberate divergence from the SDK convention.

Fix 1.1

The finding is fixed by documenting the native sentinel in `IAxelarApp.rescueFunds`, including that `address(0)` is treated as an ERC-20 and reverts through the safe-transfer path.

[Go back to Findings Summary](#)

I2: Inverted service availability documentation

Impact:	Info	Likelihood:	N/A
Type:	Code quality	Discoverer:	Human
Target:	Utils.sol		

Description

Utils.checkServiceAvailability documents address(0) as an opt-out value for optional services, but the implementation reverts when service is address(0). The helper therefore enforces a mandatory dependency, and the current documentation can cause future optional integrations to become undeployable on chains where the service is absent.

Listing 39. Excerpt from Utils

```
28 /// @notice Revert with `ServiceUnavailable` if `service` is the zero  
29 address.  
29 /// @dev Use in constructors to guard optional services (e.g. Permit2) that  
30 may not be deployed  
30 /// on every network. Pass `address(0)` to opt out of the service on  
31 unsupported chains.  
31 /// @param service Address of the service to verify.  
32 function checkServiceAvailability(address service) internal pure {  
33     if (service == address(0)) revert ServiceUnavailable();
```

The only current call site uses the helper for mandatory multicall_, while optional Permit2 is stored as a possibly-zero immutable and gated at call time.

Recommendation

Update the @dev comment to state that checkServiceAvailability is for mandatory services. Remove the address(0) opt-out guidance and the Permit2 example, and direct optional-service integrations to the store-then-gate pattern used by AxelarPermit2. Consider normalizing the constructor zero-address error taxonomy.

Fix 1.1

The finding is fixed by correcting the `Utils.checkServiceAvailability` documentation to describe a mandatory dependency, and by recording the constructor error taxonomy in the `ServiceUnavailable` documentation in `IAxelarApp`.

[Go back to Findings Summary](#)

I3: Deprecated ITS transfer overload

Impact:	Info	Likelihood:	N/A
Type:	Code quality	Discoverer:	Human
Target:	AxelarApp.sol, IInterchainTokenService.sol		

Description

AxelarApp sends every `SendKind.InterchainToken` path through the six-argument `interchainTransfer` overload.

Listing 40. Excerpt from `AxelarApp._sendInterchainToken`

```

183 IInterchainTokenService(interchainTokenService).interchainTransfer{value:
    interchainSend.gasValue}{
184     interchainSend.tokenId,
185     params.destinationChain,
186     interchainSend.destinationAddress,
187     amount,
188     metadata,
189     interchainSend.gasValue
190 };

```

The local minimal `IInterchainTokenService` mirror exposes this overload as the `send` primitive and omits the upstream v2.2.0 NatSpec deprecation notice.

Listing 41. Excerpt from `IInterchainTokenService`

```

5 /// @notice Minimal ITS surface used by the send path ('callAndSend').
  'interchainTransfer' performs
6 /// the send; 'registeredTokenAddress' resolves the ERC20 to approve from
  the 'tokenId'. The inherited
7 /// 'InterchainTokenExecutable' already wires the receive side.
8 /// @dev Local minimal mirror of 'axelar-network/interchain-token-
  service@2.2.0'; parity pinned by

```

Listing 42. Excerpt from IInterchainTokenService

```
11 function interchainTransfer(  
12     bytes32 tokenId,  
13     string calldata destinationChain,  
14     bytes calldata destinationAddress,  
15     uint256 amount,  
16     bytes calldata metadata,  
17     uint256 gasValue  
18 ) external payable;
```

Current behavior is correct because the overload is still implemented and the metadata format matches the contract-call send path. The issue is forward compatibility with upgradeable live ITS deployments: `InterfaceParity.t.sol` pins only the compile-time dependency selector, so it cannot detect a live ITS upgrade that removes this deprecated selector. If that happens, all `InterchainToken callAndSend` sends revert atomically before token transfer.

Recommendation

Migrate ITS contract-call sends to `callContractWithInterchainToken(tokenId, destinationChain, destinationAddress, amount, destinationPayload)` and pass gas as `msg.value`. Handle empty `destinationPayload` explicitly. At minimum, copy the upstream deprecation notice into `IInterchainTokenService` and document the compatibility assumption.

Fix 1.1

The finding is fixed by copying the upstream deprecation notice into the local `IInterchainTokenService` mirror. The send path continues to use the deprecated overload.

[Go back to Findings Summary](#)

Report Revision 1.1

Revision Team

Revision team is the same as in [Report Revision 1.0](#).

System Overview

The architecture remained unchanged since [Report Revision 1.0](#).

Trust Model

The trust model remained unchanged since [Report Revision 1.0](#).

Appendix A: How to cite

Cite this document as:

[ack3](#), Security Review | AxelarApp: Contracts, August 13, 2026.



Thank you.

COMPANY	ACK3 LTD
WEB	ack3.ai
X	@ack3ai
MAIL	hello@ack3.ai